Assignment 2: Architectural MVC Patterns and Frameworks

Following up from the class lecture notes and this week's tutorial on the MVC architecture.

Background on MVC and Frameworks

A challenge with the MVC architectural pattern is that there are different interpretations of this pattern. This becomes clear when particular frameworks are leveraged. An example of this is the Java Swing framework that facilitates Java user interfaces.

The Cash Register Example

In this assignment you are going to leverage some of the Cash Register components were defined in the in-class exercise 2 and during lectures. The ones of particular interest are: *CashRegister*, *Display*, and *Scanner*.

The Swing MVC Example

In the GitHub classroom I have created a GitHub assignment (please visit https://classroom.github.com/a/57HVEcop) based on a GitHub template that contains a classical MVC model that reflects the Java Swing framework. The example is a simple example of Swing interface that takes as input a person's name and will display it back again. The files are names *Model*, *View*, and *Controller* that correspond to the MVC components in the pattern.

Submission Instructions

Some parts of this assignment require a document to be submitted while others require coding in the GitHub classroom repository.

Deliverables

1. Do some investigation into the Swing framework and write a short paragraph describing the purpose of the Swing framework. Submit a class diagram of the components of Swing.
2. Look through the example code in the GitHub repository and explain how this example implements the MVC pattern. How does it differ from the conventional MVC pattern described in the lectures?
3. In the repository there is another Java file that is named Scanner. This Class emulates the scanning of a product by generating a UPC code. If you press on the Scan button you will see that it prints out the "12345" code to the console. For this assignment, modify the Scanner so that each time the *Scan* button is pressed, it randomly selects a UPC code from a file containing product information with the following format.

UPC Code	Product	Price
12345	Coffee	$8.32
67890	Muffin	2.50

In addition, consider coding a “Cash Register” application leveraging the Swing MVC model with the following design.

- Model (CashRegister): Reads product information from the file and maintains a list of scanned items. Provides methods to add an item by UPC and calculate the running subtotal.
- Controller: Receives scan events from the Scanner, tells the CashRegister to add the scanned item, and updates the View.
- View (Display): Shows a scrollable list of all scanned items with their prices, and displays the running subtotal at the bottom.

When the TA executes your main Java file, they should be able to repeatedly press the Scan button and see multiple products appear in the scrollable list, with the subtotal updating automatically.

4. Create a sequence diagram of your design for the scenario presented in question 3 representing a single press of the scan button.